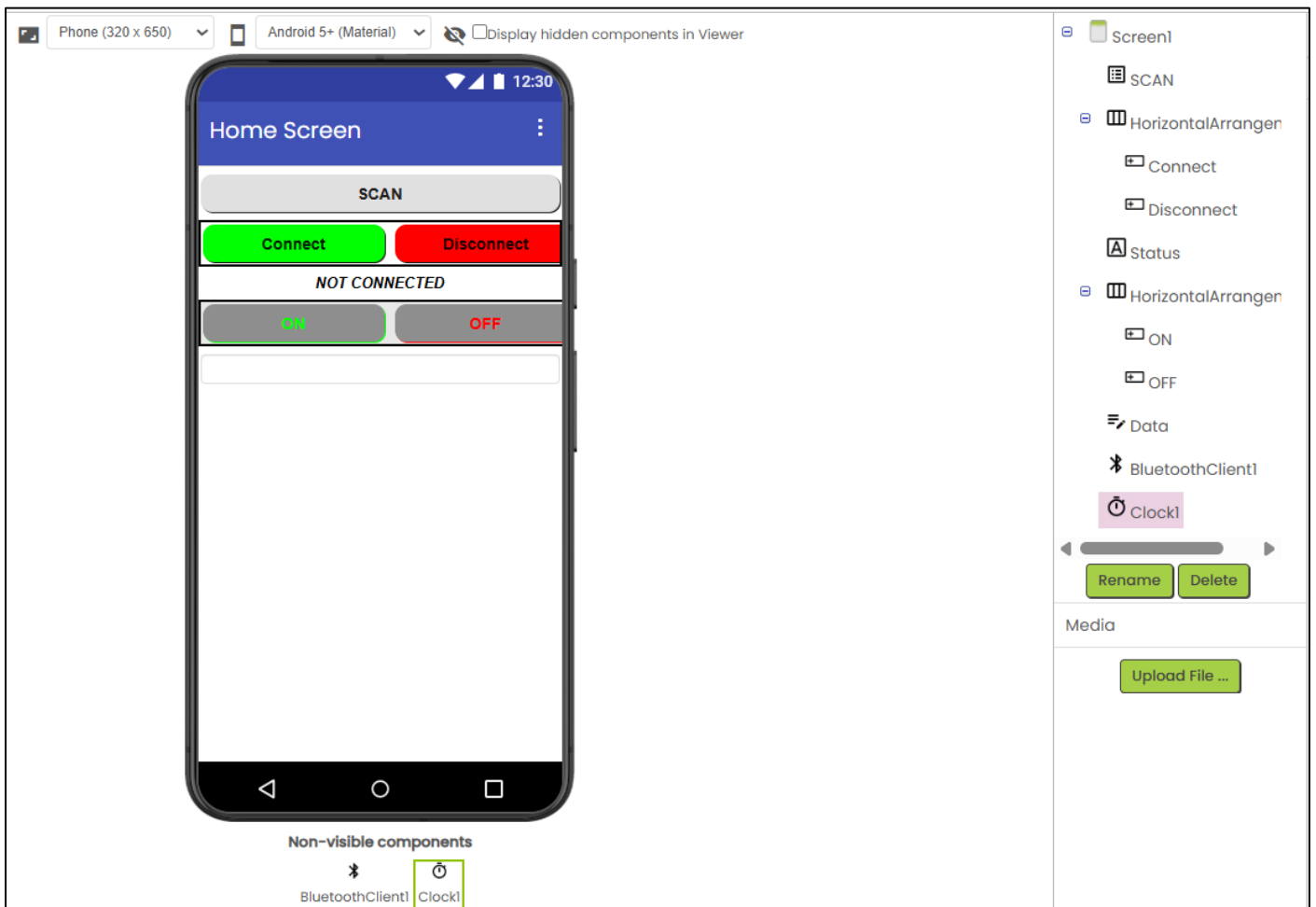
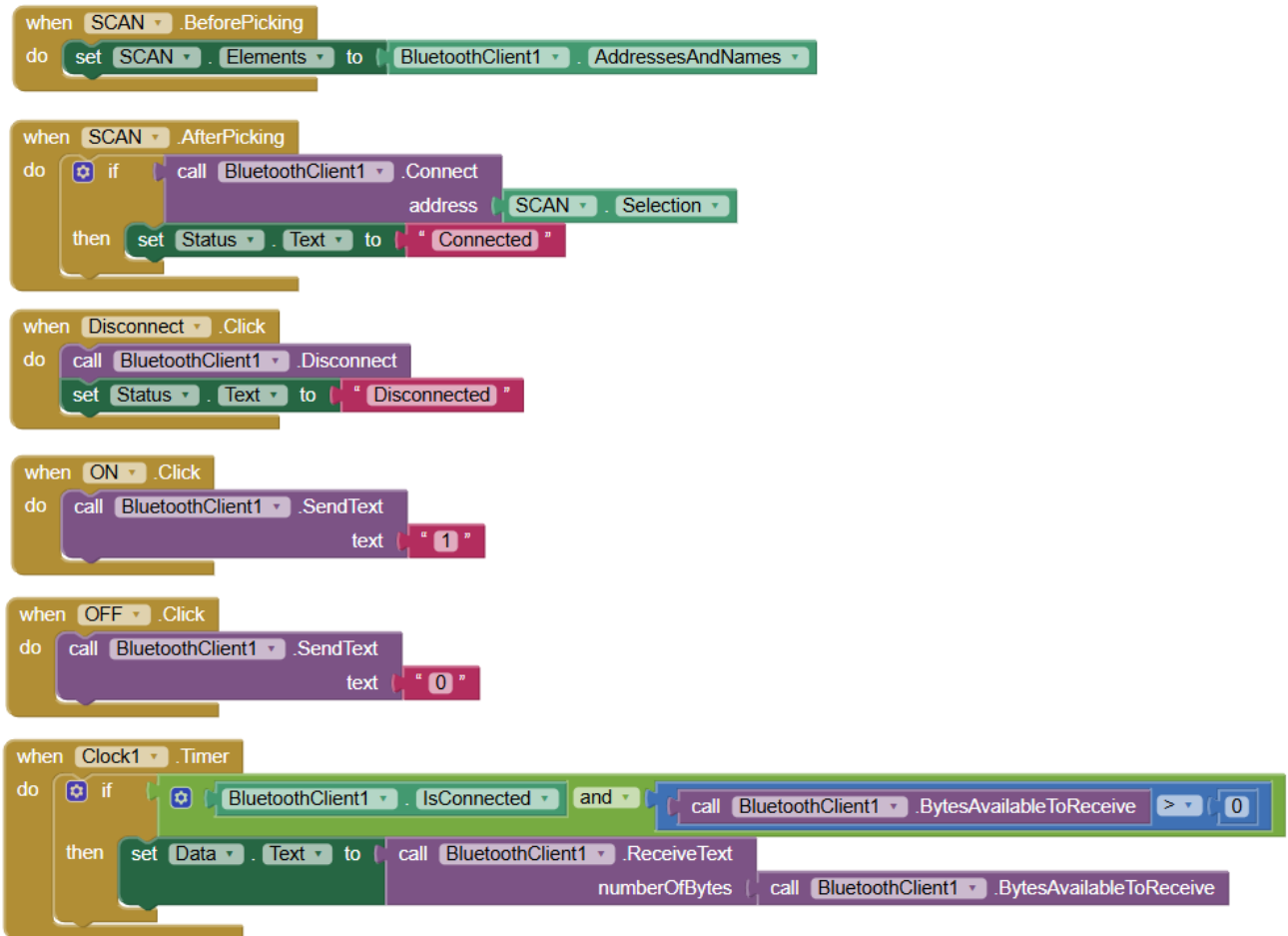




lloT_P1.ino

```

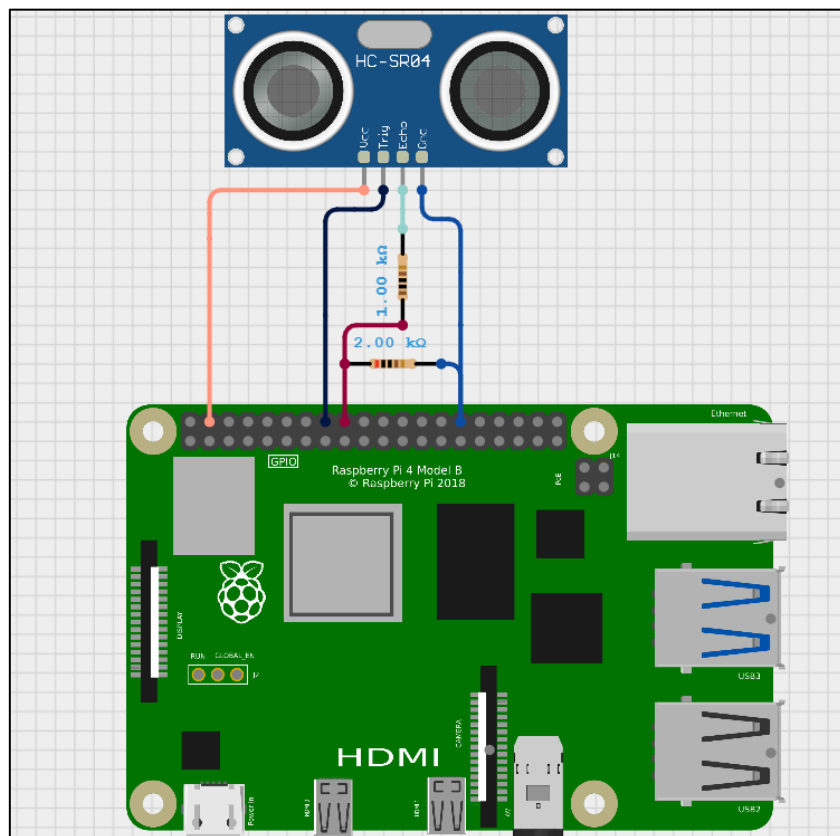
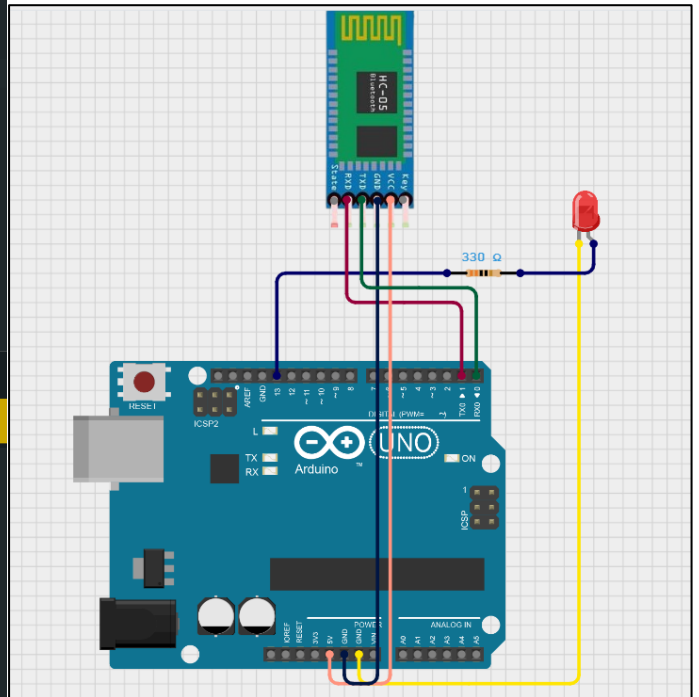
1 char inputByte;
2 void setup() {
3     Serial.begin(9600);
4     pinMode(13, OUTPUT);
5 }
6
7 void loop() {
8     while (Serial.available() > 0) {
9         inputByte = Serial.read();
10        Serial.println(inputByte);
11        if (inputByte == '1') {
12            digitalWrite(13, HIGH);
13        } else if (inputByte == '0') {
14            digitalWrite(13, LOW);
15        }
16    }
17 }
18

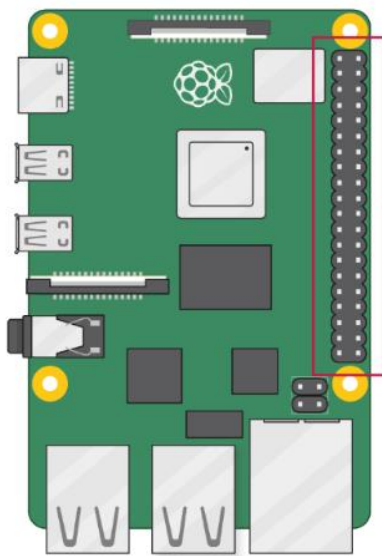
```

Output Serial Monitor X

Not connected. Select a board and a port to connect automatically.

0
1
0
1
0
1
0
1
0
1
0





3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)

```
import RPi.GPIO as GPIO
import time
```

```
GPIO.setmode(GPIO.BCM)
```

```
PIN_TRIGGER = 23
PIN_ECHO = 24
```

```
GPIO.setup(PIN_TRIGGER, GPIO.OUT)
GPIO.setup(PIN_ECHO, GPIO.IN)
```

```
GPIO.output(PIN_TRIGGER, GPIO.LOW)
```

```
print("Waiting for sensor to settle")
time.sleep(2)
```

```
try:
```

```
    while True:
```

```
        # Send trigger pulse
```

```
        GPIO.output(PIN_TRIGGER, GPIO.HIGH)
```

```
        time.sleep(0.00001)
```

```
        GPIO.output(PIN_TRIGGER, GPIO.LOW)
```

```
        # Wait for echo start
```

```
        while GPIO.input(PIN_ECHO) == 0:
            pulse_start_time = time.time()
```

```
        # Wait for echo end
```

```
        while GPIO.input(PIN_ECHO) == 1:
            pulse_end_time = time.time()
```

```
        # Calculate distance
```

```
        pulse_duration = pulse_end_time - pulse_start_time
```

```
        distance = round(pulse_duration * 17150, 2)
```

```
print("Distance:", distance, "cm")
```

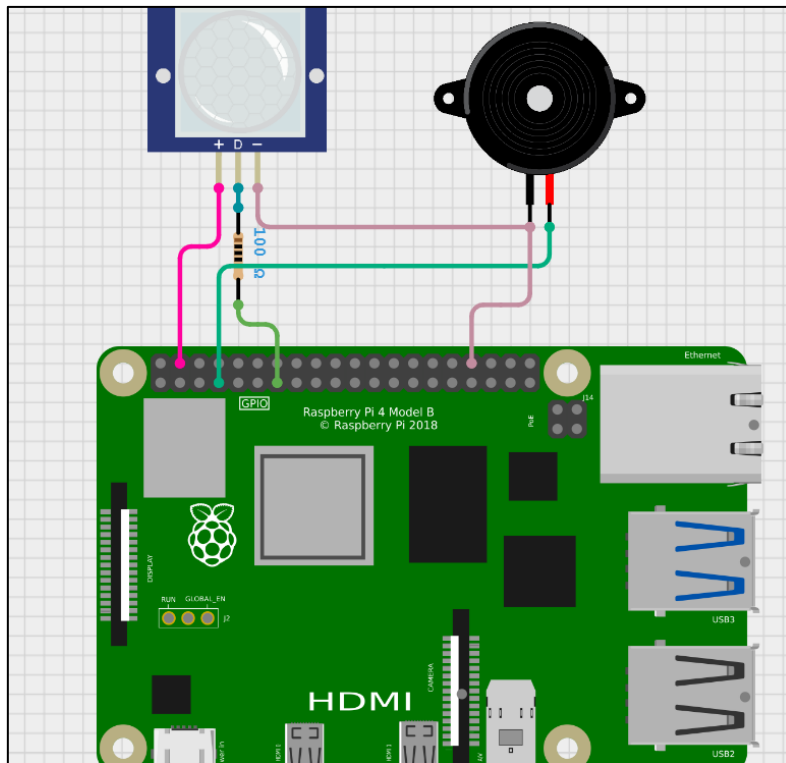
```
time.sleep(0.5) # small delay to avoid flooding output
```

```
except KeyboardInterrupt:
```

```
    print("Measurement stopped by user")
```

```
finally:
```

```
    GPIO.cleanup()
```



```
import RPi.GPIO as GPIO
```

```
import time
```

```
sensor = 11
```

```
buzzer = 7
```

```
GPIO.setmode(GPIO.BOARD)
```

```
GPIO.setup(sensor, GPIO.IN) # PIR sensor
```

```
GPIO.setup(buzzer, GPIO.OUT) # Buzzer
```

```
GPIO.output(buzzer, False)
```

```
while True:
```

```
    if GPIO.input(sensor):
```

```
        GPIO.output(buzzer, True)
```

```
        print("Motion detected")
```

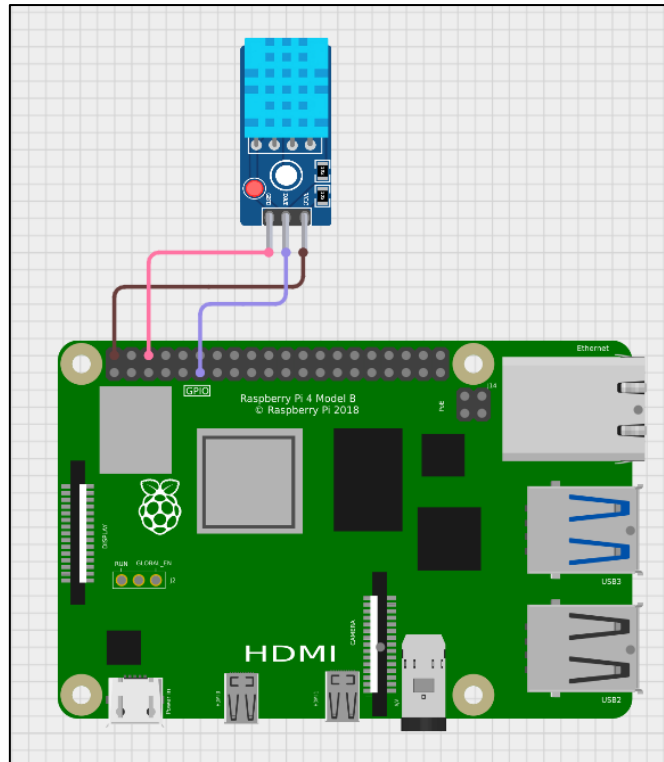
```
        time.sleep(1)
```

```
    else:
```

```
        print("Motion not detected")
```

```
        GPIO.output(buzzer, False)
```

```
        time.sleep(1)
```



```
import time
import board
import adafruit_dht

# Initial the dht device, with data pin connected to:
dhtDevice = adafruit_dht.DHT11(board.D17)

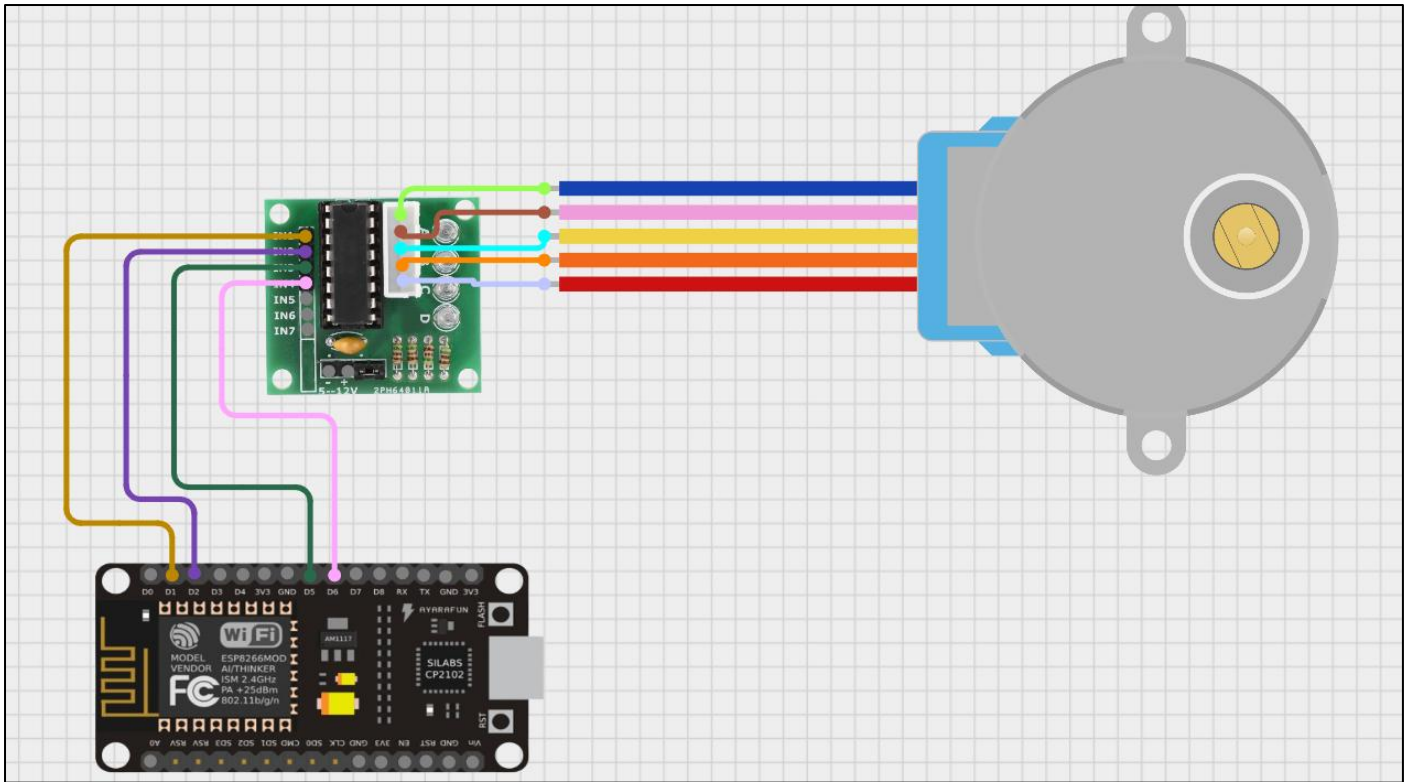
while True:
    try:
        # Print the values to the serial port
        temperature_c = dhtDevice.temperature
        humidity = dhtDevice.humidity

        print( "Temp: {:.1f} C  Humidity: {}% ".format(temperature_c, humidity))

    except RuntimeError as error:
        # Errors happen fairly often, DHT's are hard to read, just keep going
        print(error.args[0])
        time.sleep(2.0)
        continue

    except Exception as error:
        dhtDevice.exit()
        raise error

    time.sleep(2.0)
```



```
#include <AccelStepper.h>
```

```
const int stepsPerRevolution = 2048; // change this to fit the number of steps per revolution
```

```
// ULN2003 Motor Driver Pins
```

```
#define IN1 5
#define IN2 4
#define IN3 14
#define IN4 12
```

```
// initialize the stepper library
```

```
AccelStepper stepper(AccelStepper::HALF4WIRE, IN1, IN3, IN2, IN4);
```

```
void setup() {
```

```
  // initialize the serial port
  Serial.begin(115200);
```

```
  // set the speed and acceleration
```

```
  stepper.setMaxSpeed(500);
  stepper.setAcceleration(100);
```

```
  // set target position
```

```
  stepper.moveTo(stepsPerRevolution);
```

```
}
```

```
void loop() {
```

```
  // check current stepper motor position to invert direction
```

```
  if (stepper.distanceToGo() == 0){
    stepper.moveTo(-stepper.currentPosition());
    Serial.println("Changing direction");
  }
```

```
  // move the stepper motor (one step at a time)
```

```
  stepper.run();
```

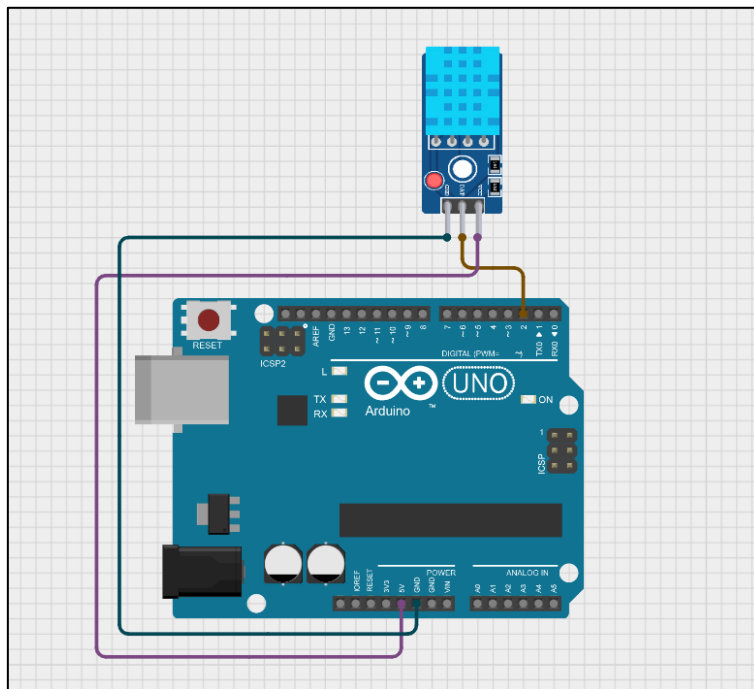
```
}
```

```
#include <ESP8266WiFi.h>
```

```
const char* ssid = "SuperNova";  
const char* password = "Willow2023";
```

```
void setup() {  
  Serial.begin(115200);  
  WiFi.begin(ssid, password);  
  
  Serial.print("Connecting to WiFi");  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }  
  
  Serial.println("\nConnected to WiFi");  
  Serial.print("IP Address: ");  
  Serial.println(WiFi.localIP());  
}
```

```
void loop() {  
}
```



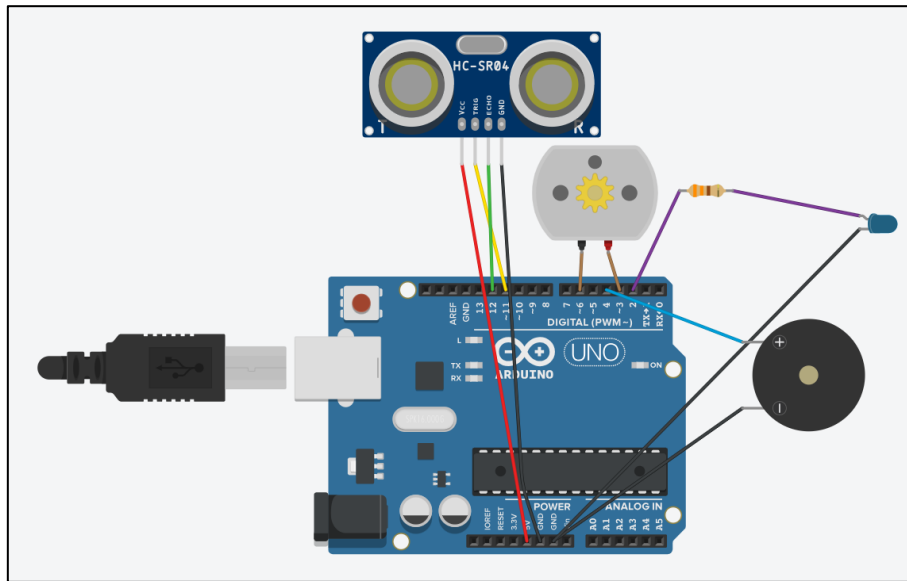
```
#include <DHT.h>  
#define DHTPIN 2  
#define DHTTYPE DHT11 // DHT 11 (AM2302)  
DHT dht(DHTPIN, DHTTYPE);
```

```
float hum;  
float temp;  
void setup()  
{  
  Serial.begin(9600);  
  dht.begin();  
}  
String hum1;
```

```

String temp1;
void loop()
{
  hum = dht.readHumidity();
  temp= dht.readTemperature();
  hum1 = String(hum);
  temp1 = String(temp);
  Serial.print("Humidity: ");
  Serial.print(hum1);
  Serial.print(", ");
  Serial.print("Temp: ");
  Serial.println(temp1);
  delay(2000);
}

```



```

int trigPin = 11;
int echoPin = 12;

int led = 2;
int buzz = 4;

int motorp = 3;
int motorn = 6;

long duration, cm, inches;

void setup() {
  Serial.begin(9600);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  pinMode(buzz, OUTPUT);
  pinMode(led, OUTPUT);

  pinMode(motorp, OUTPUT);
  pinMode(motorn, OUTPUT);
}

```

```
void loop() {  
  
    // Trigger ultrasonic pulse  
    digitalWrite(trigPin, LOW);  
    delayMicroseconds(5);  
  
    digitalWrite(trigPin, HIGH);  
    delayMicroseconds(10);  
  
    digitalWrite(trigPin, LOW);  
  
    // Read echo time  
    duration = pulseIn(echoPin, HIGH);  
  
    // Convert to distance in cm  
    cm = (duration / 2) * 0.034;  
  
    // Print distance  
    Serial.print(cm);  
    Serial.print(" cm");  
    Serial.println();  
  
    // Default motor forward  
    digitalWrite(motorp, HIGH);  
  
    // Condition 1: Object within 50 cm  
    if (cm < 50) {  
        digitalWrite(led, HIGH);  
    }  
  
    delay(250);  
  
    // Condition 2: Object within 40 cm  
    if (cm < 40) {  
        digitalWrite(buzz, HIGH);  
        digitalWrite(motorp, LOW);  
    }  
  
    delay(250);  
  
    // Condition 3: Object within 20 cm  
    if (cm < 20) {  
        digitalWrite(led, HIGH);  
        digitalWrite(buzz, HIGH);  
        digitalWrite(motorn, HIGH);  
    }  
  
    delay(1000);  
  
    // Reset outputs  
    digitalWrite(led, LOW);  
    digitalWrite(buzz, LOW);  
    digitalWrite(motorn, LOW);  
    digitalWrite(motorp, HIGH);  
}
```